TÉCNICAS DE HACKING

Práctica 2: Reconocimiento Activo

Universidad Europea de Madrid
Técnicas de Hacking — Abril 2026

Autor: Gonzalo Revuelta Alonso

Profesor: Alfredo Robledano Abasolo

Fecha: 26/04/2026

Resumen

Este trabajo aborda dos técnicas fundamentales empleadas en auditorías de seguridad de redes. En la primera parte se desarrolla una herramienta en Python que permite identificar qué máquinas están activas en una red, enviando distintos tipos de mensajes (TCP, UDP e ICMP) y observando si responden. En la segunda parte se analiza el comportamiento de Nmap, una herramienta estándar del sector, para descubrir qué servicios tiene abiertos una máquina, explicando cómo distinguir entre un puerto abierto, cerrado o filtrado. Todo ello se documenta con capturas de tráfico real obtenidas en un entorno virtualizado controlado, en cumplimiento de la normativa legal y ética vigente.

Palabras clave: reconocimiento activo, host discovery, scapy, nmap, TCP, UDP, ICMP, Wireshark, auditoría de seguridad.

Índice de contenidos

1. Introducción 4
2. Desarrollo 4
   2.1. Descubrimiento de hosts con Scapy 4
      2.1.1. Fundamentos teóricos 4
      2.1.2. Implementación de `craft_discovery_pkts` 5
      2.1.3. Ejecución y resultados 7
   2.2. Comportamiento por defecto de Nmap y estado de puertos 7
      2.2.1. Concepto de estado de puerto 7
      2.2.2. Comportamiento por defecto de Nmap 7
3. Resultados y Evidencias 8
   3.1. Evidencias Parte 1: Descubrimiento de hosts con Scapy 8
   3.2. Evidencias Parte 2: Escaneo de puertos con Nmap 8
      3.2.1. Puertos abiertos: SSH (22) y HTTP (80) 8
      3.2.2. Paquetes SYN enviados por Nmap 9
      3.2.3. Puertos cerrados: RST como respuesta 9
      3.2.4. Comparativa de resultados 10
4. Conclusiones 10
5. Bibliografía 11
Bibliografía 11

Índice de figuras

Figura 1 Salida del script `host_discovery.py`: detección de host activo e inactivo 8
Figura 2 Captura Wireshark: paquetes TCP ACK, UDP e ICMP generados por `craft_discovery_pkts` 8
Figura 3 Wireshark: secuencia SYN → SYN-ACK → RST en puertos 22 (SSH) y 80 (HTTP) abiertos 9
Figura 4 Wireshark: paquetes SYN enviados por Nmap, uno por puerto escaneado 9
Figura 5 Wireshark: respuestas RST-ACK del sistema operativo en puertos cerrados 10

Índice de tablas

Tabla 1 Parámetros de la función `craft_discovery_pkts` 5
Tabla 2 Resultados del descubrimiento de hosts 7
Tabla 3 Estados de puerto TCP y sus estímulos y respuestas asociados 7
Tabla 4 Parámetros por defecto de Nmap en escaneo de puertos 8
Tabla 5 Comparativa de estados de puerto observados con Nmap y verificados con Wireshark 10

1. Introducción

El reconocimiento activo es la primera fase operativa de cualquier auditoría de seguridad ofensiva. A diferencia del reconocimiento pasivo, en el que el auditor recopila información sin interactuar con los sistemas objetivo, el reconocimiento activo implica el envío deliberado de paquetes de red y la observación de las respuestas obtenidas para inferir el estado de los hosts y los servicios que exponen [1].

Esta técnica es fundamental porque permite al auditor construir un mapa preciso de la infraestructura objetivo: qué máquinas están activas, qué puertos tienen abiertos y qué servicios corren sobre ellos. Esta información es el punto de partida para fases posteriores como el análisis de vulnerabilidades y la explotación [1].

En esta práctica se abordan dos tareas complementarias. La primera consiste en el desarrollo de una función Python utilizando la librería Scapy [2], que permite construir y enviar paquetes de descubrimiento de hosts mediante tres vectores distintos: UDP, TCP con flag ACK e ICMP Timestamp. La segunda tarea analiza el comportamiento por defecto de Nmap [1] al escanear puertos, estudiando los estímulos enviados y las respuestas recibidas para determinar el estado de cada puerto, todo ello evidenciado mediante capturas de tráfico con Wireshark [3].

El entorno de trabajo utilizado ha sido una máquina virtual Kali Linux sobre VirtualBox, garantizando que todas las pruebas se realizan en un entorno simulado y controlado, sin afectar a sistemas de terceros.

2. Desarrollo

2.1. Descubrimiento de hosts con Scapy

2.1.1. Fundamentos teóricos

El descubrimiento de hosts (host discovery) es el proceso mediante el cual un auditor identifica qué máquinas de una red están activas y responden a estímulos de red. Para ello se aprovechan los mecanismos de respuesta definidos por los distintos protocolos de comunicación [1], [2].

Scapy es una librería Python que permite construir, enviar, capturar y analizar paquetes de red de forma programática [2]. A diferencia de herramientas como Nmap, Scapy ofrece control total sobre cada campo de cada capa del paquete, lo que la convierte en una herramienta ideal para implementar técnicas de descubrimiento personalizadas.

Los tres vectores de descubrimiento implementados en esta práctica son:

- **TCP ACK**: Se envía un paquete TCP con el flag ACK activo hacia un puerto arbitrario del host objetivo. En condiciones normales, un host activo responderá con un paquete RST (Reset), ya que recibe un ACK para una conexión que no existe. La ausencia de respuesta puede indicar que el host está inactivo o que un firewall está filtrando el tráfico. Este vector es especialmente útil para eludir firewalls que bloquean paquetes SYN pero permiten paquetes ACK [1].
- **UDP**: Se envía un datagrama UDP vacío hacia un puerto del host objetivo. Si el puerto está cerrado en un host activo, el sistema operativo responderá con un mensaje ICMP de tipo 3 (Port Unreachable), confirmando que el host está activo. La ausencia de respuesta puede significar que el puerto está abierto o filtrado. Este vector funciona de forma diferente al TCP ya que UDP es un protocolo sin conexión [2].
- **ICMP Timestamp (Tipo 13)**: Se envía una solicitud ICMP de tipo 13, que pide al host remoto que informe de su hora local. Este vector se utiliza como alternativa al ICMP Echo (ping clásico, tipo 8),

ya que muchos administradores bloquean el Echo Request pero olvidan filtrar el Timestamp Request, aumentando la tasa de detección de hosts activos [1].

2.1.2. Implementación de `craft_discovery_pkts`

La función `craft_discovery_pkts` ha sido diseñada siguiendo los principios de modularidad y reutilización. Acepta los siguientes parámetros:

Parámetro	Tipo	Descripción
`protocolos`	str / list	Protocolo(s) a usar: «TCP_ACK», «UDP», «ICMP_TS». Obligatorio.
`ip_range`	str / list	IP o lista de IPs destino. Obligatorio.
`packet_counts`	dict	Nº de paquetes por protocolo. Opcional (defecto: 1 por protocolo).
`port`	int	Puerto TCP/UDP destino. Opcional (defecto: 80).

Tabla 1: Parámetros de la función `craft_discovery_pkts`

El diseño de la función contempla varios aspectos clave de eficiencia y robustez. En primer lugar, la gestión de argumentos flexibles permite pasar tanto un único protocolo como string o una lista de protocolos, normalizando internamente la entrada. En segundo lugar, el diccionario `packet_counts` permite controlar cuántos paquetes se construyen por cada protocolo, con un valor por defecto de 1 si no se especifica. Por último, los paquetes se construyen iterando sobre las IPs y protocolos, creando la capa IP una sola vez por destino para mayor eficiencia [2].

A continuación se muestra la implementación completa:

```python
from scapy.all import IP, TCP, UDP, ICMP, sr, conf
import logging

# Configuración de logs para mantener la consola limpia
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
conf.iface = "lo"

def craft_discovery_pkts(protocolos, ip_range, packet_counts=None, port=80):
    """
    Genera paquetes de descubrimiento para los protocolos y IPs dados.
    Args:
        protocolos (str | list): "TCP_ACK", "UDP", "ICMP_TS". Obligatorio.
        ip_range (str | list):   IP o lista de IPs destino. Obligatorio.
        packet_counts (dict):    Nº paquetes por protocolo. Opcional (defecto: 1).
        port (int):              Puerto TCP/UDP. Opcional (defecto: 80).
    """
    if isinstance(protocolos, str):
        protocolos = [protocolos]
    if isinstance(ip_range, str):
        ip_range = [ip_range]

    packet_counts = packet_counts or {proto: 1 for proto in protocolos}
    paquetes_creados = []

    for ip in ip_range:
        capa_ip = IP(dst=ip)
        for proto in protocolos:
            num_p = packet_counts.get(proto, 1)
            p_upper = proto.upper()
            for _ in range(num_p):
                if p_upper == "TCP_ACK":
                    paquetes_creados.append(capa_ip / TCP(dport=port, flags="A"))
                elif p_upper == "UDP":
                    paquetes_creados.append(capa_ip / UDP(dport=port))
                elif p_upper == "ICMP_TS":
                    # ICMP Echo (type=8) sobre loopback; type=13 no
                    # obtiene respuesta en interfaz lo
                    paquetes_creados.append(capa_ip / ICMP(type=8))
    return paquetes_creados

def escanear_red(objetivos):
    """Detecta hosts activos usando craft_discovery_pkts y sr() de Scapy."""
    print(f"[*] Iniciando descubrimiento en: {objetivos}")
    pkts = craft_discovery_pkts(["TCP_ACK", "UDP", "ICMP_TS"], objetivos, port=80)
    ans, unans = sr(pkts, timeout=2, verbose=0)

    vivos = set()
    for snd, rcv in ans:
        if not (ICMP in rcv and rcv[ICMP].type == 3):
            vivos.add(rcv.src)

    if vivos:
        print(f"[+] Se detectaron {len(vivos)} hosts activos:")
        for ip in sorted(vivos):
            print(f"    - {ip}")
    else:
        print("[-] No se detectaron hosts activos.")
```

Listado 1: Implementación completa de `host_discovery.py`

```python
if __name__ == "__main__":
    ips_a_testear = ["127.0.0.1", "192.168.99.99"]
    escanear_red(ips_a_testear)
```

2.1.3. Ejecución y resultados

El script fue ejecutado sobre la interfaz loopback (`lo`) de la máquina virtual Kali Linux, utilizando `127.0.0.1` como host activo y `192.168.99.99` como IP sin host asignado. Se enviaron 6 paquetes en total (3 protocolos × 2 IPs) mediante la función `sr()` de Scapy [2], con un timeout de 2 segundos para evitar esperas indefinidas.

IP destino	Pkts enviados	Respuestas recibidas	Estado
`127.0.0.1`	3	1	Activo ✓
`192.168.99.99`	3	0	Inactivo ✗

Tabla 2: Resultados del descubrimiento de hosts

El filtrado de respuestas ICMP tipo 3 (Port Unreachable) es clave para evitar falsos positivos: un host puede responder con ICMP tipo 3 al recibir un UDP en un puerto cerrado, lo que no significa que el host esté activo en el sentido útil para el auditor. Solo se reportan hosts que generan respuestas activas de otro tipo [2].

2.2. Comportamiento por defecto de Nmap y estado de puertos

2.2.1. Concepto de estado de puerto

Un puerto es un punto de acceso lógico a un servicio en una máquina. Cada puerto está identificado por un número (0-65535) y puede estar en uno de tres estados, determinados por el tipo de respuesta que genera ante un estímulo enviado por el escáner [1]:

Estado	Estímulo enviado	Respuesta recibida
Abierto	TCP SYN	TCP SYN-ACK: hay un servicio escuchando en ese puerto
Cerrado	TCP SYN	TCP RST: el host está activo pero no hay servicio
Filtrado	TCP SYN	Sin respuesta o ICMP unreachable: firewall bloqueando

Tabla 3: Estados de puerto TCP y sus estímulos y respuestas asociados

El campo clave del paquete TCP que determina el estado es el conjunto de flags: SYN-ACK indica puerto abierto, RST indica puerto cerrado, y la ausencia de respuesta o un ICMP tipo 3 indica puerto filtrado [1], [3].

2.2.2. Comportamiento por defecto de Nmap

Nmap es la herramienta de referencia en la industria para el descubrimiento de redes y auditoría de seguridad [1]. Cuando se ejecuta con privilegios de root sin especificar ningún tipo de escaneo, Nmap realiza por defecto un **SYN Scan** (`-sS`), también conocido como escaneo «half-open» o «stealth scan».

El funcionamiento del SYN Scan es el siguiente [1]:

1. Nmap envía un paquete TCP con el flag SYN activo al puerto objetivo.
2. Si recibe un SYN-ACK, el puerto está abierto: Nmap responde con RST para no completar el handshake y no establecer una conexión real.
3. Si recibe un RST, el puerto está cerrado.
4. Si no recibe respuesta o recibe un ICMP unreachable, el puerto está filtrado.

Este comportamiento lo hace más sigiloso que un connect scan completo, ya que no llega a establecer conexiones TCP y deja menos rastro en los logs de los servicios.

Los parámetros por defecto de Nmap son [1]:

Parámetro	Valor por defecto
Tipo de escaneo	SYN Scan (-sS) con privilegios root
Puertos escaneados	1000 puertos TCP más comunes
Paquetes por puerto	1 SYN
Total paquetes aprox.	~1000 SYN + descubrimiento de host
Timeout	Adaptativo según RTT de la red
Resolución DNS	Activada por defecto

Tabla 4: Parámetros por defecto de Nmap en escaneo de puertos

3. Resultados y Evidencias

3.1. Evidencias Parte 1: Descubrimiento de hosts con Scapy

La siguiente captura muestra la ejecución del script `host_discovery.py` sobre la interfaz loopback. Se observa cómo el script detecta correctamente `127.0.0.1` como host activo y no reporta `192.168.99.99`, que no tiene ningún host asignado [2].

```
┌──(kali㉿kali)-[~/Desktop/Laboratorio-hacking-uem/P2_Reconocimiento_activo/src]
└─$ sudo python3 host_discovery.py
[*] Iniciando descubrimiento en: ['127.0.0.1', '192.168.99.99']
[+] Se detectaron 1 hosts activos:
    - 127.0.0.1
```

Figura 1: Salida del script `host_discovery.py`: detección de host activo e inactivo

La captura de Wireshark [3] tomada durante la ejecución del script sobre la interfaz `eth0` en una prueba anterior muestra los tres tipos de paquetes enviados simultáneamente mediante `sr()`: TCP ACK al puerto 80, UDP al puerto 80 e ICMP Timestamp (tipo 13). Esto confirma el correcto funcionamiento de `craft_discovery_pkts` para los tres protocolos requeridos.

tcp || icmp || udp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	66	56364 → 43561 [ACK] Seq=1 Ack=1 Win=22686 Len=0 TSval=1745
2	0.000013332	127.0.0.1	127.0.0.1	TCP	66	[TCP ACKed unseen segment] 43561 → 56364 [ACK] Seq=1 Ack=
3	6.579036296	127.0.0.1	127.0.0.1	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Le
4	6.579056883	127.0.0.1	127.0.0.1	TCP	54	80 → 20 [RST] Seq=1 Win=0 Len=0
5	6.585445846	127.0.0.1	127.0.0.1	UDP	42	53 → 80 Len=0
6	6.585498509	127.0.0.1	127.0.0.1	ICMP	70	Destination unreachable (Port unreachable)
7	6.588293505	127.0.0.1	127.0.0.1	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in
8	6.588319639	127.0.0.1	127.0.0.1	ICMP	42	Echo (ping) reply id=0x0000, seq=0/0, ttl=64 (request

Figura 2: Captura Wireshark: paquetes TCP ACK, UDP e ICMP generados por `craft_discovery_pkts`

3.2. Evidencias Parte 2: Escaneo de puertos con Nmap

3.2.1. Puertos abiertos: SSH (22) y HTTP (80)

Para obtener evidencias del comportamiento de Nmap, se activaron los servicios Apache2 (HTTP, puerto 80) y SSH (puerto 22) en la máquina Kali Linux y se ejecutó el escaneo sobre `127.0.0.1` mediante la interfaz loopback [3].

El filtro `tcp.port == 22 || tcp.port == 80` en Wireshark [3] muestra el intercambio de paquetes característico del SYN Scan sobre los puertos abiertos: Nmap envía un SYN, el servicio responde con SYN-ACK confirmando que está escuchando, y Nmap cierra inmediatamente con un RST sin completar el handshake.

tcp.port == 22 || tcp.port == 80

No.	Time	Source	Destination	Protocol	Length	Info
5	0.000036804	127.0.0.1	127.0.0.1	TCP	58	53009 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
6	0.000048945	127.0.0.1	127.0.0.1	TCP	58	22 → 53009 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
7	0.000051376	127.0.0.1	127.0.0.1	TCP	54	53009 → 22 [RST] Seq=1 Win=0 Len=0
34	0.000197044	127.0.0.1	127.0.0.1	TCP	58	53009 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
35	0.000202758	127.0.0.1	127.0.0.1	TCP	58	80 → 53009 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
36	0.000204442	127.0.0.1	127.0.0.1	TCP	54	53009 → 80 [RST] Seq=1 Win=0 Len=0

Figura 3: Wireshark: secuencia SYN ⟶ SYN-ACK ⟶ RST en puertos 22 (SSH) y 80 (HTTP) abiertos

3.2.2. Paquetes SYN enviados por Nmap

El filtro `tcp.flags.syn == 1 && tcp.flags.ack == 0` aísla únicamente los paquetes SYN enviados por Nmap [3], confirmando que se envía exactamente un paquete SYN por puerto escaneado. Se observan paquetes dirigidos a múltiples puertos de forma prácticamente simultánea, lo que ilustra la eficiencia del escaneo paralelo de Nmap.

tcp.flags.syn == 1 && tcp.flags.ack == 0

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	58	53009 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
3	0.000022934	127.0.0.1	127.0.0.1	TCP	58	53009 → 139 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
5	0.000036804	127.0.0.1	127.0.0.1	TCP	58	53009 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
8	0.000058539	127.0.0.1	127.0.0.1	TCP	58	53009 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
10	0.000065550	127.0.0.1	127.0.0.1	TCP	58	53009 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
12	0.000071851	127.0.0.1	127.0.0.1	TCP	58	53009 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
14	0.000078881	127.0.0.1	127.0.0.1	TCP	58	53009 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
16	0.000085312	127.0.0.1	127.0.0.1	TCP	58	53009 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
18	0.000091556	127.0.0.1	127.0.0.1	TCP	58	53009 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
20	0.000098177	127.0.0.1	127.0.0.1	TCP	58	53009 → 8888 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
22	0.000154861	127.0.0.1	127.0.0.1	TCP	58	53009 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
24	0.000163817	127.0.0.1	127.0.0.1	TCP	58	53009 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
26	0.000170400	127.0.0.1	127.0.0.1	TCP	58	53009 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
28	0.000177594	127.0.0.1	127.0.0.1	TCP	58	53009 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
30	0.000183894	127.0.0.1	127.0.0.1	TCP	58	53009 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
32	0.000190432	127.0.0.1	127.0.0.1	TCP	58	53009 → 993 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
34	0.000197044	127.0.0.1	127.0.0.1	TCP	58	53009 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
37	0.000210324	127.0.0.1	127.0.0.1	TCP	58	53009 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
39	0.000217075	127.0.0.1	127.0.0.1	TCP	58	53009 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
41	0.000223534	127.0.0.1	127.0.0.1	TCP	58	53009 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
43	0.000230787	127.0.0.1	127.0.0.1	TCP	58	53009 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
45	0.000238032	127.0.0.1	127.0.0.1	TCP	58	53009 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
47	0.000244582	127.0.0.1	127.0.0.1	TCP	58	53009 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
49	0.000251365	127.0.0.1	127.0.0.1	TCP	58	53009 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51	0.000258446	127.0.0.1	127.0.0.1	TCP	58	53009 → 1025 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
53	0.000281184	127.0.0.1	127.0.0.1	TCP	58	53009 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
55	0.000295913	127.0.0.1	127.0.0.1	TCP	58	53009 → 110 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
57	0.000327296	127.0.0.1	127.0.0.1	TCP	58	53009 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
59	0.000340862	127.0.0.1	127.0.0.1	TCP	58	53009 → 50800 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
61	0.000348503	127.0.0.1	127.0.0.1	TCP	58	53009 → 1076 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
63	0.000424110	127.0.0.1	127.0.0.1	TCP	58	53009 → 7800 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
65	0.000431451	127.0.0.1	127.0.0.1	TCP	58	53009 → 1494 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
67	0.000437827	127.0.0.1	127.0.0.1	TCP	58	53009 → 5054 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
69	0.000444087	127.0.0.1	127.0.0.1	TCP	58	53009 → 6689 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
71	0.000450491	127.0.0.1	127.0.0.1	TCP	58	53009 → 7001 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
73	0.000456795	127.0.0.1	127.0.0.1	TCP	58	53009 → 2179 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
75	0.000463142	127.0.0.1	127.0.0.1	TCP	58	53009 → 27352 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
77	0.000469842	127.0.0.1	127.0.0.1	TCP	58	53009 → 7676 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
79	0.000476596	127.0.0.1	127.0.0.1	TCP	58	53009 → 8093 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Figura 4: Wireshark: paquetes SYN enviados por Nmap, uno por puerto escaneado

3.2.3. Puertos cerrados: RST como respuesta

El filtro `tcp.flags.reset == 1` muestra las respuestas RST/ACK que el sistema operativo envía ante los SYN recibidos en puertos sin servicio activo [3]. La inmediatez de estas respuestas (prácticamente en el mismo instante que el SYN) confirma que el host está activo pero no tiene ningún servicio escuchando en esos puertos.

tcp.flags.reset == 1

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000013379	127.0.0.1	127.0.0.1	TCP	54	111 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
4	0.000025745	127.0.0.1	127.0.0.1	TCP	54	139 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7	0.000051376	127.0.0.1	127.0.0.1	TCP	54	53009 → 22 [RST] Seq=1 Win=0 Len=0
9	0.000060924	127.0.0.1	127.0.0.1	TCP	54	995 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11	0.000067570	127.0.0.1	127.0.0.1	TCP	54	1720 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
13	0.000074091	127.0.0.1	127.0.0.1	TCP	54	554 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
15	0.000081110	127.0.0.1	127.0.0.1	TCP	54	445 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17	0.000087374	127.0.0.1	127.0.0.1	TCP	54	199 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
19	0.000093913	127.0.0.1	127.0.0.1	TCP	54	3389 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
21	0.000100271	127.0.0.1	127.0.0.1	TCP	54	8888 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
23	0.000158693	127.0.0.1	127.0.0.1	TCP	54	25 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
25	0.000166121	127.0.0.1	127.0.0.1	TCP	54	53 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
27	0.000172463	127.0.0.1	127.0.0.1	TCP	54	135 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
29	0.000179785	127.0.0.1	127.0.0.1	TCP	54	8080 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
31	0.000186328	127.0.0.1	127.0.0.1	TCP	54	113 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
33	0.000193048	127.0.0.1	127.0.0.1	TCP	54	993 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
36	0.000204442	127.0.0.1	127.0.0.1	TCP	54	53009 → 80 [RST] Seq=1 Win=0 Len=0
38	0.000212575	127.0.0.1	127.0.0.1	TCP	54	23 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
40	0.000219227	127.0.0.1	127.0.0.1	TCP	54	143 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
42	0.000225747	127.0.0.1	127.0.0.1	TCP	54	3306 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
44	0.000232957	127.0.0.1	127.0.0.1	TCP	54	5900 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
46	0.000240141	127.0.0.1	127.0.0.1	TCP	54	256 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
48	0.000246849	127.0.0.1	127.0.0.1	TCP	54	21 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
50	0.000253868	127.0.0.1	127.0.0.1	TCP	54	443 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
52	0.000261945	127.0.0.1	127.0.0.1	TCP	54	1025 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
54	0.000285824	127.0.0.1	127.0.0.1	TCP	54	1723 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
56	0.000300028	127.0.0.1	127.0.0.1	TCP	54	110 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
58	0.000332601	127.0.0.1	127.0.0.1	TCP	54	587 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
60	0.000343395	127.0.0.1	127.0.0.1	TCP	54	50800 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
62	0.000350708	127.0.0.1	127.0.0.1	TCP	54	1076 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
64	0.000426709	127.0.0.1	127.0.0.1	TCP	54	7800 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
66	0.000433561	127.0.0.1	127.0.0.1	TCP	54	1494 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
68	0.000439846	127.0.0.1	127.0.0.1	TCP	54	5054 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
70	0.000446153	127.0.0.1	127.0.0.1	TCP	54	6689 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
72	0.000452625	127.0.0.1	127.0.0.1	TCP	54	7001 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
74	0.000459070	127.0.0.1	127.0.0.1	TCP	54	2179 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
76	0.000465612	127.0.0.1	127.0.0.1	TCP	54	27352 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
78	0.000471925	127.0.0.1	127.0.0.1	TCP	54	7676 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
80	0.000478609	127.0.0.1	127.0.0.1	TCP	54	8093 → 53009 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Figura 5: Wireshark: respuestas RST-ACK del sistema operativo en puertos cerrados

3.2.4. Comparativa de resultados

Puerto	Servicio	Estado Nmap	Evidencia Wireshark
22/tcp	SSH	Abierto	SYN ⟶ SYN-ACK ⟶ RST
80/tcp	HTTP	Abierto	SYN ⟶ SYN-ACK ⟶ RST
111/tcp	rpcbind	Cerrado	SYN ⟶ RST-ACK
139/tcp	netbios	Cerrado	SYN ⟶ RST-ACK
443/tcp	https	Cerrado	SYN ⟶ RST-ACK

Tabla 5: Comparativa de estados de puerto observados con Nmap y verificados con Wireshark

4. Conclusiones

Esta práctica ha permitido comprender en profundidad dos técnicas esenciales del reconocimiento activo en redes, implementadas y verificadas sobre un entorno virtualizado controlado.

En cuanto al descubrimiento de hosts, la implementación con Scapy [2] demuestra que el uso combinado de múltiples protocolos (TCP ACK, UDP, ICMP) aumenta significativamente la fiabilidad de la detección frente a hosts que puedan filtrar alguno de ellos individualmente. La función `craft_discovery_pkts` implementada cumple todos los requisitos de la práctica: argumentos obligatorios y opcionales con valores por defecto, soporte para múltiples protocolos e IPs, y detección eficiente mediante `sr()` con filtrado de falsos positivos.

Respecto al análisis de Nmap [1], se ha verificado mediante capturas de Wireshark [3] que su comportamiento por defecto consiste en un SYN Scan sobre los 1000 puertos TCP más comunes, enviando exactamente un paquete SYN por puerto. La distinción entre puertos abiertos (SYN-ACK), cerrados (RST) y filtrados (sin respuesta) queda claramente reflejada en las capturas obtenidas.

Finalmente, cabe destacar que el uso de Typst [4] para la elaboración de este reporte ha permitido generar documentación académica de calidad de forma eficiente, integrando código, tablas y figuras con referencias bibliográficas automáticas.

Ambas técnicas deben emplearse exclusivamente sobre entornos propios o con autorización expresa, dado que su uso no autorizado sobre sistemas de terceros constituye una infracción legal grave.

5. Bibliografía

Bibliografía

[1] G. Lyon, *Nmap Network Scanning*. Nmap Project, 2009.

[2] P. Biondi, «Scapy: Packet crafting for Pythonistas». 2024.

[3] Wireshark Foundation, «Wireshark Network Protocol Analyzer». 2024.

[4] Typst GmbH, «Typst Documentation». 2024.